Requirements, models, run files, and settings

Source: `vignettes/articles/under-the-hood.Rmd`

Reference notes that do not fit in a getting-started page.

Requirements

- **R 4.1 or newer.**
- **Disk:** about 3 GB for the Python environment, plus 0.3 to 4 GB per model in the Hugging Face cache.
- **Python:** nothing to install. `reticulate` uses a Python 3.10 to 3.13 it finds on the machine, or downloads one. torch, transformers, and peft are installed automatically on first use.
- **GPU training on NVIDIA:** the only thing you install yourself is the NVIDIA driver, from nvidia.com/en-us/drivers. Driver 580 or newer gets the CUDA 13 build of torch, 570 or newer gets CUDA 12.8, and older drivers get CUDA 12.6. The CUDA Toolkit and cuDNN are not needed; torch wheels bundle their own CUDA libraries. Check your driver with `nvidia-smi`.
- **Apple Silicon:** trains on the GPU through Metal with no setup.
- **No GPU:** training runs on the CPU. Fine for the 135M and 360M models, slow beyond that.

Run `dragon_check()` after installing. It reports the device it will train on, and if that is the CPU on a machine with an NVIDIA GPU it says why (no driver, a driver too old for the installed torch, or a CPU-only torch build) and prints the one-line fix.

Where runs are stored. By default a run's files (adapter, checkpoints, data, logs) go under a `dragonfarm_runs` folder inside a session temp directory, so a fresh R session never writes to your working directory or home filespace on its own; that folder disappears when the session ends. For runs you want to keep, set a real location once per project, before training anything:

or set the `DRAGONFARM_RUNS_DIR` environment variable, or pass `runs_dir =` to `dragon_train()`, `dragon_bundle()`, `dragon_app()`, and the rest directly. See `?dragon_runs_dir`.

What you get from a run

File	Written by	Contents
`config.json`	R	Everything the trainer needs.
`data/train.jsonl`, `data/eval.jsonl`	R	Rows in chat format.
`status.json`	Python	State, device, parameter counts, final metrics.
`progress.jsonl`	Python	Loss, learning rate, and ETA per logging step.
`adapter/`	Python	The LoRA adapter, loadable with `peft`.
`checkpoints/`	Python	The last two checkpoints, for resume.
`eval.json`, `samples.json`	Python	Held-out loss and sample generations.
`merged/`	Python	After `dragon_merge()`: a standalone model.

Models that work well

`HuggingFaceTB/SmolLM2-135M-Instruct`	135M	Apache 2.0	no	2 GB, or CPU
`HuggingFaceTB/SmolLM2-360M-Instruct`	360M	Apache 2.0	no	3 GB
`Qwen/Qwen2.5-0.5B-Instruct`	0.5B	Apache 2.0	no	3 GB
`google/gemma-3-1b-it`	1B	Gemma	yes	5 GB
`meta-llama/Llama-3.2-1B-Instruct`	1.2B	Llama 3.2	yes	5 GB
`Qwen/Qwen2.5-1.5B-Instruct`	1.5B	Apache 2.0	no	7 GB
`HuggingFaceTB/SmolLM2-1.7B-Instruct`	1.7B	Apache 2.0	no	8 GB

`dragon_presets()` returns this table. Any other causal language model on the Hugging Face Hub works too. For gated models, accept the license on the Hub and set `HF_TOKEN` in the R session.

How it works

R never imports torch. It writes a run directory and launches `python -m dragonfarm.train` as a subprocess with `processx`. The trainer is Hugging Face `transformers` with `peft` for LoRA and a prompt-masking collator so only the reply tokens contribute to the loss. Progress comes back through files, which is what lets the Shiny app poll it and lets a run outlive the R session.

The Python side (`inst/python`) is also its own installable package (`pip install ./inst/python`, soon `pip install dragonfarm` once it's on PyPI): a `dragonfarm` command (`check`, `train`, `generate`, `pack`) and a `dragonfarm.api` module for reading a run directory from plain Python, no R required. A run started from R can be inspected or continued from Python and back, since both read and write the exact same files.

Variable	Effect
`DRAGONFARM_PYTHON`	Use this interpreter instead of the one reticulate builds. It must already have the packages from `dragon_python_requirements()`.
`DRAGONFARM_TORCH_INDEX`	Windows only. `auto` (default) selects the CUDA wheel index matching your NVIDIA driver on first Python use. Set to `""` to use PyPI's CPU build, or to another index URL.
`DRAGONFARM_RUNS_DIR`	Where runs are stored. Defaults to `dragonfarm_runs` under a session temp directory; see Requirements for a persistent location.
`HF_TOKEN`	Hugging Face token for gated models.
`LLAMA_CPP_DIR`	A llama.cpp checkout, for `dragon_export_gguf()`.

Development

```
devtools::test()                                  # unit tes
Sys.setenv(DRAGONFARM_INTEGRATION = "true")
devtools::test(filter = "integration")            # trains S
```

The Python side has its own tests: `PYTHONPATH=inst/python python inst/python/tests/run.py` (or `python -m pytest inst/python/tests` if pytest is installed).

On this page

Models that work well

How it works

Environment variables

Development